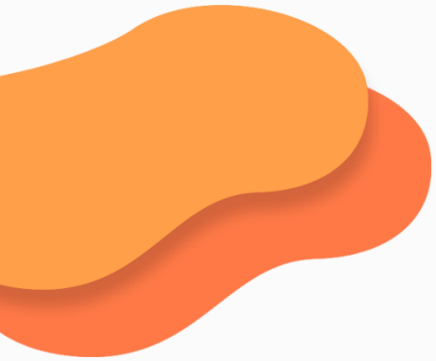


Tutorial 6

Constructing a Code Generator

Programming Languages & Compilers Spring 2025

School of Computer Science, Wuhan University



PART ONE

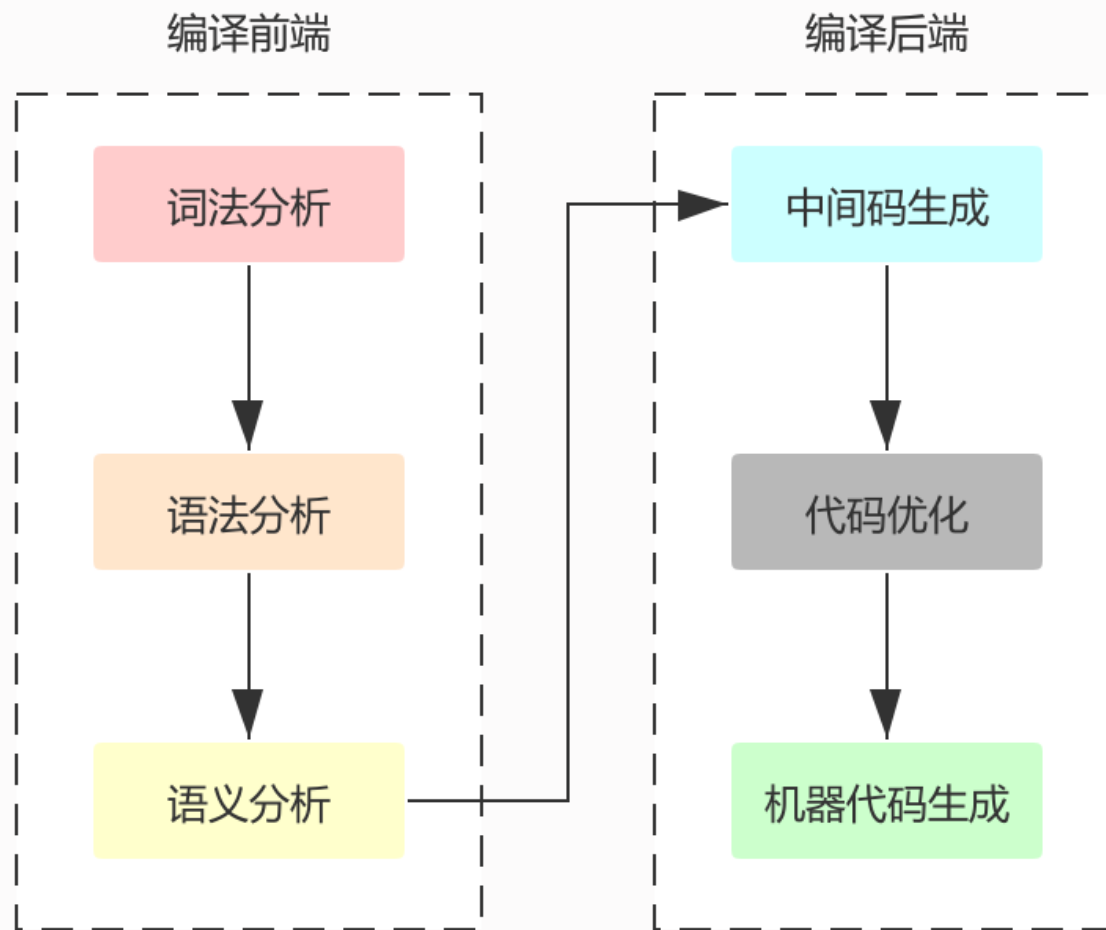
01

实验内容&RISC-V

实验内容 & RISC-V

- 前面的实验完成了前端部分
- 完整的编译器（如LLVM）需先完成AST到IR的转换，并进行代码优化，再到汇编

E.g. [Compiler Explorer](#)



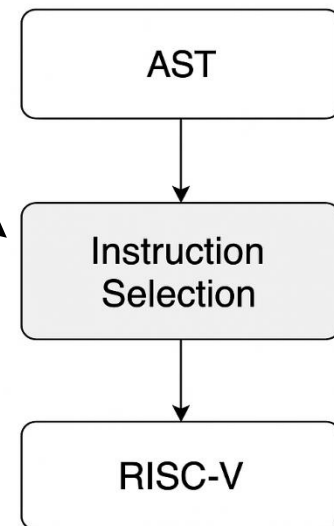
实验内容 & RISC-V

- 本次实验实现从**AST到RISC-V**（不考虑优化）
- 参考riscvcard.pdf了解具体需要的**汇编指令**和**寄存器类型**

REGISTER NAME, USE, CALLING CONVENTION

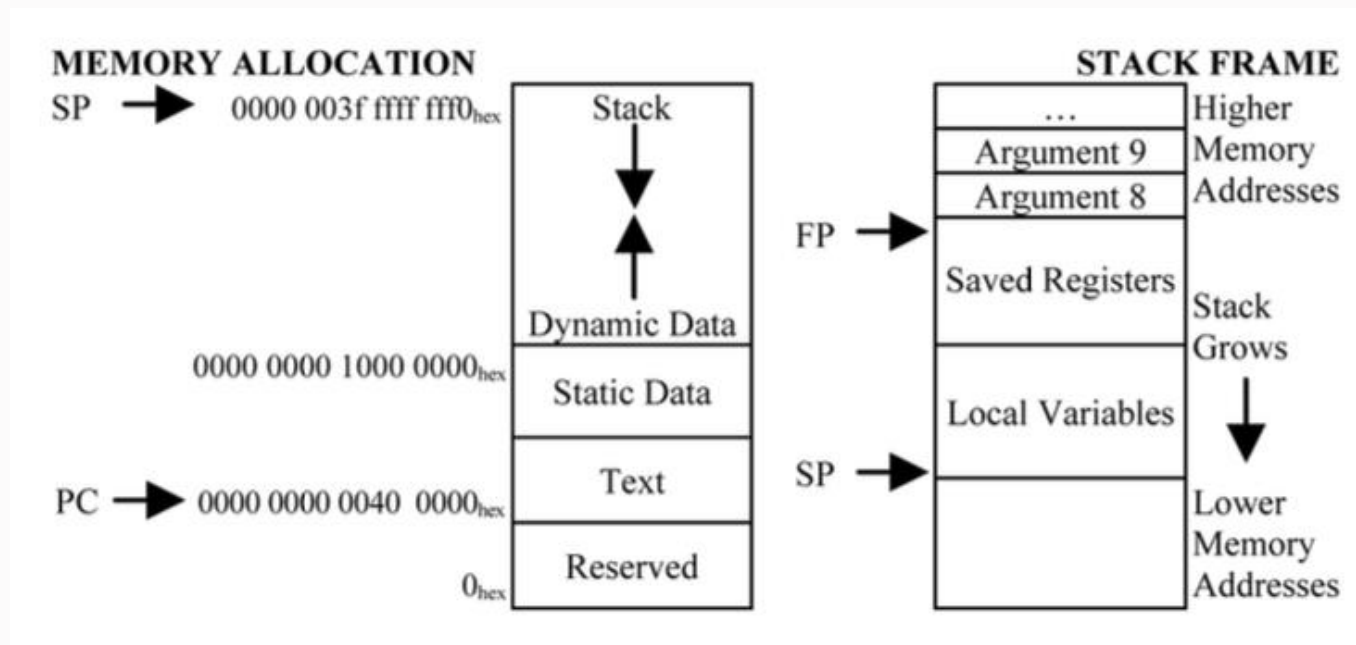
④

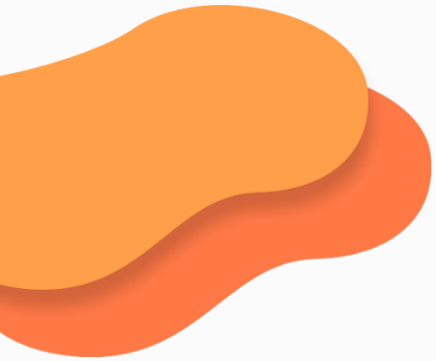
REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/Return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP Temporaries	Caller
f8-f9	fs0-fs1	FP Saved registers	Callee
f10-f11	fa0-fa1	FP Function arguments/Return values	Caller
f12-f17	fa2-fa7	FP Function arguments	Caller
f18-f27	fs2-fs11	FP Saved registers	Callee
f28-f31	ft8-ft11	$R[rd] = R[rs1] + R[rs2]$	Caller



实验内容 & RISC-V

- 用 x8 作为帧指针(fp) 来固定指向当前函数的**栈帧基址**
- 使用 **a0 寄存器**作为表达式计算结果的默认存放寄存器（函数返回值也约定放在 a0）
- 避免寄存器分配:使用栈来保存局部变量和临时数据





PART TWO

02

Task 1: Binop -> Riscv

Binop -> Riscv

C++ source #1

Save/Load + Add new... Vim CppInsights Quick-bench C++

```
1 // Type your code here, or load an example.
2
3 int main() {
4     int a = 8 * 7;
5     a = a + 9;
6     return 0;
7 }
```

RISC-V rv64gc clang 20.1.0 (Editor #1)

RISC-V rv64gc clang 20.1.0 Compiler options...

Output... Filter... Libraries Overrides + Add new... Add tool...

```
1 main:
2     addi    sp, sp, -32
3     sd      ra, 24(sp)
4     sd      s0, 16(sp)
5     addi    s0, sp, 32
6     li      a0, 0
7     sw      a0, -20(s0)
8     li      a1, 56
9     sw      a1, -24(s0)
10    lw      a1, -24(s0)
11    addiw    a1, a1, 9
12    sw      a1, -24(s0)
13    ld      ra, 24(sp)
14    ld      s0, 16(sp)
15    addi    sp, sp, 32
16    ret
```

Output (0/0) RISC-V rv64gc clang 20.1.0 - 685ms (3262B) ~184 lines filtered Compiler License

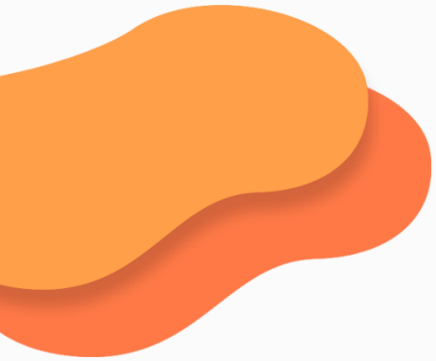
Binop -> Riscv

- Int 常数类型: 直接将常数值加载到寄存器。RISC-V 提供了 li (load immediate) 伪指令, 将一个立即数载入寄存器。
- Binop 考虑处理三种情况:
 - Num Binop Num E.g. $2 + 7$
 - Num Binop Num Binop Num E.g. $2 + 6 * 9$
 - (Num Binop Num) Binop (Num Binop Num) E.g. $(2 + 6) * (8 + 7)$

Binop -> Riscv

- 基本流程: 计算左操作数 -> 保存左值 -> 计算右操作数 -> 运算合成结果
- 两种方案保存:
 - 将左值保存在临时寄存器中(例如t0),再计算右值到另一个寄存器(例如t1)
 - 采用栈,在计算左值后将其压栈保存,计算右值后再弹栈取出左值进行运算

Hint: 参考simpl_test1.in生成的test1.s



PART THREE

03

Task2: Let -> Riscv

Let -> Riscv

- 增加对**变量和作用域**的支持,使编译器可以处理let绑定和变量引用
- 栈帧与变量地址：当进入一个新的let块，需在栈上分配新的空间保存局部变量的值（调整栈指针fp的offset）
- 符号表：在编译阶段需要一个映射（符号表）跟踪当前作用域中每个变量名的栈偏移或寄存器位置（env）

Let -> Riscv

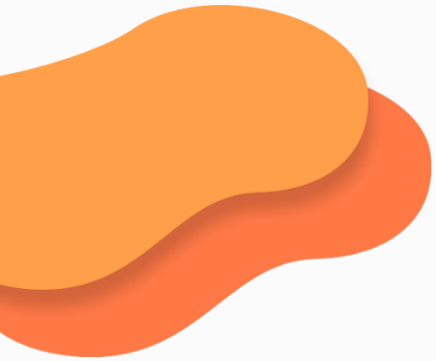
- 简化逻辑，fp固定不动，变量x偏移可以按照进入let的次序决定
- 固定帧具体而言：在程序一开始一次性分配最大所需栈，然后fp = sp固定

```
.text
.globl main
main:
    addi sp, sp, -STACK_SIZE    # 申请栈空间
    mv fp, sp                  # 设置帧指针
    ... (这里是compile_expr生成的代码) ...
    mv a0, zero                # (如果需要返回值，这里假设返回0表示程序成功结束)
    addi sp, sp, STACK_SIZE    # 回收栈
    ret
```

Let -> Riscv

- 编译e1得到其值在a0
- 在栈上分配空间保存其值 (addi sp, sp, -8)
- 将变量名x映射到新空间的位置 例如offset += 8表示又用8字节, 映射env新增 ("x", offset)
- 编译内部表达式e2, Var x按env找到偏移, 完成后**回退栈指针**释放x所占的空间 (addi sp, sp, 8)
- 恢复符号表, 弹出env中x的绑定

Hint: 参考simpl_test2.in生成的test2.s



PART FOUR

04

Task3: If -> Riscv

If -> Riscv

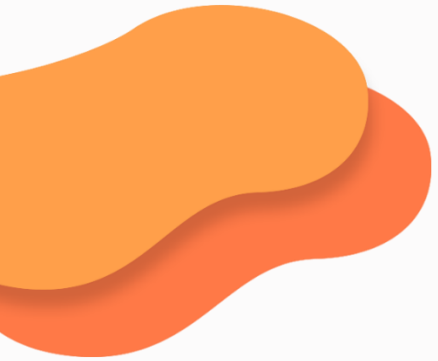
- 扩展编译器支持Bool常量和If条件表达式，正确处理程序的控制流
- 为每个if表达式生成**唯一的**标签并正确安插跳转指令
- 保证程序无论哪一种路径都会结束

```
main:
    addi    sp, sp, -32
    sd      ra, 24(sp)
    sd      s0, 16(sp)
    addi    s0, sp, 32
    li      a0, 0
    sw      a0, -20(s0)
    li      a0, 63
    sw      a0, -24(s0)
    lw      a0, -24(s0)
    li      a1, 91
    blt     a0, a1, .LBB0_2
    j       .LBB0_1
.LBB0_1:
    lw      a0, -24(s0)
    sw      a0, -20(s0)
    j       .LBB0_3
.LBB0_2:
    li      a0, 0
    sw      a0, -20(s0)
    j       .LBB0_3
.LBB0_3:
    lw      a0, -20(s0)
    ld      ra, 24(sp)
    ld      s0, 16(sp)
    addi    sp, sp, 32
    ret
```

If -> Riscv

- 思考同前面let/变量的交互：使用标签而非固定地址
- 条件的各分支内部可以定义自己的变量且作用域仅限于分支内，实现需以递归的方式编译分支表达式并各自管理环境
- 在分支中寄存器值通过a0返回结果，无残留的临时变量，无需专门清理分支的栈

Hint: 参考simpl_test3.in生成的test3.s



PART FOUR

05

Task5: Func | App ->
Riscv (optional)



Func | App -> Riscv

- 扩展编译器支持函数定义 (Func) 和函数调用 (App)
- 函数的闭包包含函数代码和绑定环境的组合体
- 编译Func(x, body)产生两部分输出：全局函数代码（储存待后续统一输出）；当前表达式计算时创建闭包的代码

Func | App -> Riscv

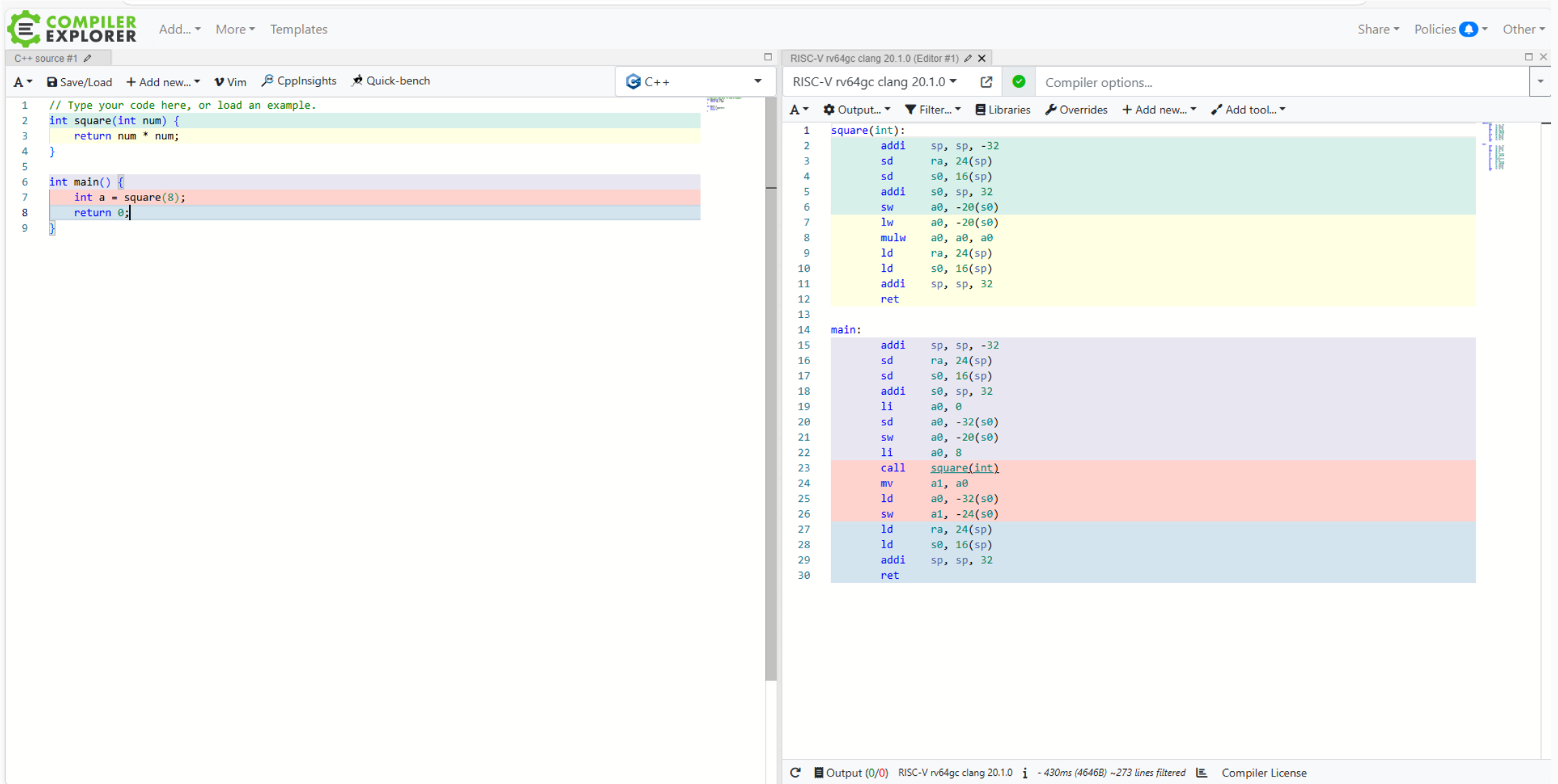
- 回顾Riscv寄存器， caller负责传递参数（a0-a7寄存器）和保存临时寄存器； callee可以自由使用临时寄存器但需保存和恢复caller保存寄存器（如s0等）
- 仅考虑函数调用一个显式参数
- 传递参数值和环境指针

REGISTER NAME, USE, CALLING CONVENTION

④

REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/Return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP Temporaries	Caller
f8-f9	fs0-fs1	FP Saved registers	Callee
f10-f11	fa0-fa1	FP Function arguments/Return values	Caller
f12-f17	fa2-fa7	FP Function arguments	Caller
f18-f27	fs2-fs11	FP Saved registers	Callee
f28-f31	ft8-ft11	$R[rd] = R[rs1] + R[rs2]$	Caller

Func | App -> Riscv



The image shows the Compiler Explorer interface with the following components:

- Header:** COMPILER EXPLORER, Add..., More, Templates, Share, Policies, Other.
- Left Panel (C++ source #1):**

```
1 // Type your code here, or load an example.
2 int square(int num) {
3     return num * num;
4 }
5
6 int main() {
7     int a = square(8);
8     return 0;
9 }
```
- Right Panel (RISC-V rv64gc clang 20.1.0 (Editor #1)):**

Compiler options...

```
1 square(int):
2     addi    sp, sp, -32
3     sd      ra, 24(sp)
4     sd      s0, 16(sp)
5     addi    s0, sp, 32
6     sw      a0, -20(s0)
7
8     lw      a0, -20(s0)
9     mulw    a0, a0, a0
10    ld      ra, 24(sp)
11    ld      s0, 16(sp)
12    addi    sp, sp, 32
13    ret
14
15 main:
16     addi    sp, sp, -32
17     sd      ra, 24(sp)
18     sd      s0, 16(sp)
19     addi    s0, sp, 32
20     li      a0, 0
21     sd      a0, -32(s0)
22     sw      a0, -20(s0)
23     li      a0, 8
24     call    square(int)
25     mv      a1, a0
26     ld      a0, -32(s0)
27     sw      a1, -24(s0)
28     ld      ra, 24(sp)
29     ld      s0, 16(sp)
30     addi    sp, sp, 32
31     ret
```
- Footer:** Output (0/0) RISC-V rv64gc clang 20.1.0 - 430ms (4646B) ~273 lines filtered Compiler License



Func | App -> Riscv

- 闭包的环境结构（仅提供一种思路）：
- 分配一块内存，大小足够存放一个指针加上所有自由变量的值
- 将函数的代码地址写入这块内存的开头
- 将该函数的自由变量当前环境中的值写入内存的后续字段
- 将指向这块内存的指针作为闭包值放入某寄存器



Func | App -> Riscv

- Func需要一个唯一的新函数标签（类似if的标签）
- 对函数体进行编译，与main形式上相同（env -> local_env + closure_env）
- 生成函数的汇编代码并保存
- 在main函数体换Func编译结果为上一步的闭包分配代码

Func | App -> Riscv

- App调用有多种方式：jalr、call等（以jalr为例）
- 将a0（闭包指针）保存到t0，a0放参数
- 使用ld t1, 0(t0)加载闭包偏移为0处内容（代码地址）到t1
- jalr ra, t1进行跳转调用
- (没做free.....)



Func | App -> Riscv

- 运行你的Riscv代码 (asm.s文件)
- possible method: Risc-V GNU Toolchain + qemu模拟器
- 较复杂，课上不讲